Analise Completa - EPBot Matrix EA v1.59 (branch v1.58-refactor)

Data: 2026-04-03
Escopo: Todos os 35 arquivos .mq5/.mqh do repositorio
Autor: Claude Opus 4.6 (analise automatizada)

Indice

1. Resumo Executivo
2. Problemas Criticos
3. Problemas de Severidade Alta
4. Problemas de Severidade Media
5. Problemas de Severidade Baixa
6. Problemas de Arquitetura
7. Resumo por Categoria

1. Resumo Executivo

O EPBot Matrix e um EA modular e bem estruturado para MT5, com sistema de estrategias plugaveis, filtros, GUI completo, persistencia de configuracao e hot reload. O codigo demonstra boa engenharia de software com separacao de responsabilidades, logging robusto e tratamento de erros.

Pontos fortes:

- Arquitetura modular limpa (Strategy/Filter pattern)
- Sistema de hot reload (input vs working parameters)
- Persistencia de estado (TradeManager.SaveState/LoadState)
- Logging orientado ao negocio com throttle inteligente
- Protecao de drawdown com multiplos modos

Resumo de problemas encontrados:

Severidade	Quantidade
Critico	4
Alto	12
Medio	18

Severidade	Quantidade
Baixo	14
Total	48

2. Problemas Criticos

CRIT-01: Variavel `openPrice` nao declarada no escopo de MonitorPartialTP

Arquivo: `Core/TradeManager.mqh`
Linhas: 646 e 705
Severidade: CRITICO

Problema: Nos blocos de TP1 (linha 646) e TP2 (linha 705), o codigo referencia `openPrice` diretamente:

```
m_logger.SavePartialTrade(ticket, dealTicket, tradeType, openPrice, realExitPrice,
```

Porem, dentro de `MonitorPartialTP()`, a variavel local se chama `m_positions[index].openPrice`. A variavel `openPrice` sozinha nao esta declarada no escopo da funcao.

Impacto: Este codigo provavelmente nem compila (erro de compilacao MQL5). Se compila por alguma razao (variavel global com mesmo nome), o valor passado ao SavePartialTrade seria incorreto, gerando registros CSV com precos de abertura errados.

Correcao:

```
// Substituir openPrice por m_positions[index].openPrice nas linhas 646 e 705
m_logger.SavePartialTrade(ticket, dealTicket, tradeType,
   m_positions[index].openPrice, realExitPrice, realVolume, realProfit, "Partial T
```

CRIT-02: Inconsistencia entre SaveTrade(finalDealProfit) e UpdateAfterTrade(finalDealProfit)

Arquivo: `EPBot_Matrix.mq5`
Linhas: 734, 738, 741
Severidade: CRITICO

Problema: O comentario na linha 736 diz "KNOWN LIMITATION" mas o changelog v1.59 diz que foi corrigido. Analise do codigo atual:

```
g_logger.SaveTrade(g_lastPositionTicket, finalDealProfit);    // linha 734 - salva d
g_logger.UpdateStats(totalPositionProfit);                    // linha 738 - stats c
g_blockers.UpdateAfterTrade(isWin, finalDealProfit);         // linha 741 - streak c
```

O `isWin` e calculado com `totalPositionProfit > 0` (correto), mas o `tradeProfit` passado ao `UpdateAfterTrade` e `finalDealProfit`, nao `totalPositionProfit`. Embora `UpdateAfterTrade` atualmente so use `isWin` para streak, o parametro `tradeProfit` nao e utilizado pela implementacao atual de `BlockerLimits::UpdateAfterTrade()` - porem o parametro existe e um futuro uso poderia introduzir inconsistencia silenciosa.

Alem disso, `SaveTrade()` salva `finalDealProfit` no CSV, enquanto `UpdateStats()` usa `totalPositionProfit`. Se a posicao teve parciais (TP1+TP2), o CSV mostra apenas o lucro do deal final, mas as stats do dia mostram o lucro total. Isso causa divergencia entre o CSV e o relatorio diario.

Impacto: O CSV de trades individuais nao reflete o lucro real total da posicao. O relatorio diario (via UpdateStats) esta correto, mas o CSV esta errado para trades com parciais.

Correcao:

```
g_logger.SaveTrade(g_lastPositionTicket, totalPositionProfit);  // usar total
g_logger.UpdateStats(totalPositionProfit);
g_blockers.UpdateAfterTrade(isWin, totalPositionProfit);       // consistencia
```

CRIT-03: Chaves mal posicionadas em MonitorPartialTP - bloco TP2

Arquivo: `Core/TradeManager.mqh`
Linhas: 686-742
Severidade: CRITICO

Problema: Analisando a estrutura de chaves no bloco TP2 (linhas 686-742), a remocao do TP fixo apos TP2 (linhas 709-741) esta DENTRO do bloco `if(ExecutePartialClose(...))`, o que e correto. Porem, ha um problema de indentacao que indica um bug potencial: o bloco `else` na linha 743 (`else { ... "Falha ao executar TP2" }`) esta alinhado com o `if` da remocao do TP, nao com o `if(ExecutePartialClose)`.

Olhando mais cuidadosamente a estrutura de chaves:

```
if(ExecutePartialClose(ticket, lotToClose, "Partial TP2", dealTicket))   // linha 68
  {
   SetTP2Executed(ticket, true);                                          // linha 688
   ...
   FetchDealRealValues(...);                                             // linha 694
   ...
   m_logger.SavePartialTrade(...);                                         // linha 705
  }                                                                     // FECHA Execu

   // REMOCAO DO TP - FORA DO if(ExecutePartialClose) !!!
   if(PositionSelectByTicket(ticket))                                     // linha 712
     {...}
  }                                                                     // esta chave
else                                                                    // linha 743
```

A chave na linha ~708 fecha o `if(ExecutePartialClose)`. A remocao do TP (linhas 712-741) fica **fora** do bloco de sucesso. Isso significa que a remocao do TP e tentada mesmo quando o TP2 **FALHA**. O `else` na linha 743 nao corresponde ao `if(ExecutePartialClose)` como esperado.

Impacto: Se `ExecutePartialClose` falhar para TP2, o codigo ainda tenta remover o TP fixo da posicao (o que pode causar comportamento inesperado) e o `else` (log de erro) pode nao ser executado.

Correcao: Reestruturar as chaves para que a remocao do TP esteja dentro do bloco de sucesso do `ExecutePartialClose`.

CRIT-04: HistorySelectByPosition usa ticket em vez de position ID

Arquivo: `EPBot_Matrix.mq5`
Linha: 704
Severidade: CRITICO

Problema:

```
if(HistorySelectByPosition(g_lastPositionTicket))
```

`HistorySelectByPosition()` espera um `position_id` (POSITION_IDENTIFIER), nao um `ticket`. Embora em muitos brokers o ticket e o position_id sejam iguais, em contas netting ou em alguns brokers eles podem diferir. O `g_lastPositionTicket` e obtido via `PositionGetTicket(i)`, que retorna POSITION_TICKET.

Na MQL5, `POSITION_TICKET` e `POSITION_IDENTIFIER` sao frequentemente iguais, mas a API exige `POSITION_IDENTIFIER` para `HistorySelectByPosition`. Internamente, `PositionGetTicket()` retorna o ticket que em contas hedging coincide com o identifier, mas formalmente o codigo deveria usar `PositionGetInteger(POSITION_IDENTIFIER)`.

Impacto: Em contas netting ou em brokers especificos, o historico de deals pode nao ser selecionado corretamente, causando falha na deteccao de fechamento de posicao (lucro = 0, streak errado).

Correcao: Armazenar o `POSITION_IDENTIFIER` quando a posicao e detectada e usar esse valor para `HistorySelectByPosition()`.

3. Problemas de Severidade Alta

HIGH-01: Candle guard usa `inp_ExitMode` em vez de working variable

Arquivo: `EPBot_Matrix.mq5`
Linhas: 756, 889, 915
Severidade: ALTA

Problema: O candle guard e FCO guard usam `inp_ExitMode` (input original):

```
if(inp_ExitMode != EXIT_VM)  // linha 756
bool isVMActive = (... inp_ExitMode == EXIT_VM);  // linha 889
if(inp_ExitMode == EXIT_FCO)  // linha 915
```

Porem, o ExitMode pode ser alterado via GUI (hot reload). Se o usuario trocar de EXIT_VM para EXIT_FCO via GUI, o candle guard continua usando o valor do input original.

Impacto: Apos hot reload do ExitMode via GUI, o comportamento de bloqueio de candle e FCO sera incorreto. O EA pode re-entrar no mesmo candle ou bloquear entradas indevidamente.

Correcao: Criar uma variavel working para ExitMode (como ja existe para MagicNumber e Slippage) e usa-la no OnTick.

HIGH-02: `inp_UsePartialTP` usado em vez de working variable

Arquivo: `EPBot_Matrix.mq5`
Linha: 1041
Severidade: ALTA

Problema:

```
if(inp_UsePartialTP)
   g_tradeManager.MonitorPartialTP(ticket);
```

O `inp_UsePartialTP` e o input original. Se o Partial TP for ativado/desativado via GUI, o `MonitorPartialTP` continuara usando o valor do input.

Tambem na linha 1382:

```
bool hasPartialTP = inp_UsePartialTP;
```

Impacto: Hot reload do Partial TP via GUI nao tem efeito. Posicoes abertas apos desativar PTP ainda terao parciais monitorados, e posicoes abertas apos ativar PTP nao terao.

Correcao: Usar `g_riskManager.IsPartialTPEnabled()` em vez de `inp_UsePartialTP`.

HIGH-03: Posicao duplicada nao tratada - apenas alerta

Arquivo: `EPBot_Matrix.mq5`
Linhas: 690-699
Severidade: ALTA

Problema: Quando detecta mais de uma posicao aberta com o mesmo magic, o codigo apenas loga um alerta:

```
if(myPositionCount > 1)
   g_logger.Log(LOG_ERROR, THROTTLE_TIME, "POSITION", "ALERTA: %d posicoes abertas.
```

Nao toma nenhuma acao corretiva. Apenas a primeira posicao encontrada e gerenciada.

Impacto: Se por qualquer motivo (race condition no OrderSend, re-envio apos timeout, posicao manual) houver multiplas posicoes, apenas uma sera gerenciada pelo trailing/breakeven/TP parcial. As demais ficarao sem gerenciamento de SL/TP.

Correcao: Alem do alerta, considerar fechar posicoes duplicadas automaticamente ou gerenciar todas.

HIGH-04: Sem protecao contra re-entrada durante gerenciamento

Arquivo: `EPBot_Matrix.mq5`
Linhas: 806-860
Severidade: ALTA

Problema: Quando o EA fecha uma posicao por horario (linhas 806-855) ou daily limit (linhas 959-994), ele usa `return` para sair do `OnTick`. Porem, se o `OrderSend` falhar (retcode != DONE), o codigo chama `ManageOpenPosition(ticket)` e retorna, mas no proximo tick, como a posicao ainda existe, o ciclo se repete. Nao ha backoff ou limite de tentativas.

Para o fechamento por horario, se o OrderSend falhar repetidamente (por exemplo, mercado fechado ou requote), o EA vai tentar fechar a cada tick indefinidamente, gerando spam de logs e requests.

Impacto: Flood de OrderSend em condicoes adversas (mercado fechado, requotes), potencialmente causando ban temporario do broker.

Correcao: Implementar backoff exponencial ou limite de tentativas para fechamentos emergenciais.

HIGH-05: Race condition entre OnTimer e OnTick na GUI

Arquivo: `EPBot_Matrix.mq5`
Linhas: 459, 608+
Severidade: ALTA

Problema: O `OnTimer()` chama `g_panel.Update()` a cada 1500ms, enquanto `OnTick()` opera no mesmo thread mas pode executar `ExecuteTrade()` e `ManageOpenPosition()` que alteram estado compartilhado (posicoes, TP flags, etc.). O MQL5 serializa OnTimer/OnTick no mesmo thread, entao nao ha race condition real. Porem, `OnChartEvent()` pode interagir com ApplyConfig() que chama setters em todos os modulos enquanto OnTick esta modificando estado.

Na pratica, MQL5 serializa todos os eventos no thread principal, entao nao ha race condition de concorrencia verdadeira. Porem, a interacao logica entre GUI (ApplyConfig aplica novos

valores) e OnTick (que pode estar no meio de um trade) merece atencao.

Impacto: Baixo em MQL5 devido a serializacao de eventos, mas a complexidade da interacao entre hot reload e trade execution e uma fonte potencial de bugs logicos.

Correcao: Documentar formalmente que todos os eventos sao serializados e que hot reload so tem efeito a partir do proximo tick.

HIGH-06: Trailing no modo SELL nao compensa spread corretamente

Arquivo: `Core/RiskManager.mqh`
Linhas: ~1560-1575 (CalculateTrailing)
Severidade: ALTA

Problema: Na compensacao de spread para trailing SELL:

```
if(posType == POSITION_TYPE_BUY)
   newSL = currentPrice - stepDistance;
   if(m_trailingCompensateSpread) newSL -= spread;
else
   newSL = currentPrice + stepDistance;
   if(m_trailingCompensateSpread) newSL += spread;
```

Para SELL, o SL esta ACIMA do preco. Compensar spread significa mover o SL para MAIS LONGE do preco (dar mais folga). Para SELL, mover para mais longe = aumentar o SL. Somar spread ao SL esta correto.

Porem, para BUY, o SL esta ABAIXO do preco. Compensar spread = mover para mais longe = diminuir o SL. Subtrair spread esta correto.

Na verdade, re-analisando: para BUY, `currentPrice` e o BID. O SL e a stop loss, que se ativa quando o BID cai ate ele. Compensar spread aqui significaria considerar que o preco real de saida e pior. A compensacao parece invertida: para BUY deveria somar spread (SL mais distante = mais seguro), para SELL deveria subtrair.

Impacto: Com `m_trailingCompensateSpread=true`, o trailing pode nao compensar o spread corretamente, deixando posicoes mais vulneraveis a serem stopadas pelo spread.

Correcao: Revisar a logica de compensacao de spread: para BUY (SL abaixo), compensar = subtrair spread do SL (mais distante). Para SELL (SL acima), compensar = somar spread ao SL (mais distante). A implementacao atual parece correta na direcao, mas merece teste unitario para confirmar.

HIGH-07: ATR handle nao recriado ao mudar periodo via hot reload

Arquivo: `Core/RiskManager.mqh`
Linhas: ~1170 (SetATRPeriod)
Severidade: ALTA

Problema:

```
void CRiskManager::SetATRPeriod(int period)
{
   int oldValue = m_atrPeriod;
   m_atrPeriod = period;
   if(oldValue != period && m_logger != NULL)
      m_logger.Log(...);
}
```

O setter apenas altera `m_atrPeriod` mas nao recria o handle do ATR (`m_handleATR`). O indicador ATR foi criado com o periodo antigo no `Init()`. Alterar `m_atrPeriod` nao tem efeito pois o handle ainda aponta para o indicador com o periodo original.

Impacto: Hot reload do periodo do ATR via GUI nao funciona. Todos os calculos baseados em ATR (SL, TP, Trailing, Breakeven) continuam usando o ATR do periodo original.

Correcao:

```
void CRiskManager::SetATRPeriod(int period)
{
   int oldValue = m_atrPeriod;
   if(oldValue == period) return;
   m_atrPeriod = period;
   // Recriar handle
   if(m_handleATR != INVALID_HANDLE)
      IndicatorRelease(m_handleATR);
   m_handleATR = iATR(m_symbol, PERIOD_CURRENT, m_atrPeriod);
   m_cachedATR = 0; m_lastATRBar = 0; // invalidar cache
   if(m_logger != NULL) m_logger.Log(...);
}
```

HIGH-08: Deletar config sem confirmacao

Arquivo: `GUI/PanelPersistence.mqh`
Severidade: ALTA

Problema: O botao "Ignorar" no banner de config salva chama `CConfigPersistence::Delete()` imediatamente, sem confirmacao. Um clique acidental deleta permanentemente a configuracao salva.

Impacto: Perda permanente de configuracao customizada do usuario.

Correcao: Adicionar dialogo de confirmacao ou pelo menos mover para uma acao menos acessivel (ex: duplo clique).

HIGH-09: PrintConfiguration() incompleto no RiskManager

Arquivo: `Core/RiskManager.mqh`
Linhas: ~1934-1993
Severidade: ALTA

Problema: O `PrintConfiguration()` e um stub que so imprime Lote e SL. Faltam: TP, Trailing, Breakeven, Partial TP, ativacoes condicionais, e compensacao de spread.

Impacto: Operador nao tem visibilidade da configuracao real do RiskManager. Dificulta diagnostico em producao.

Correcao: Completar a implementacao com todas as secoes.

HIGH-10: PrintConfiguration() ausente em BlockerFilters e BlockerLimits

Arquivo: `Core/BlockerFilters.mqh`, `Core/BlockerLimits.mqh`
Severidade: ALTA

Problema: `Blockers::PrintConfiguration()` delega para `m_filters.PrintConfiguration()` e `m_limits.PrintConfiguration()`, mas esses metodos nao existem nos submodulos. O codigo compila porque o macro `CFG_LOG` e expandido inline em `Blockers.mqh`, mas a delegacao provavelmente causa erro de compilacao.

Impacto: Erro de compilacao ou config incompleta impressa.

Correcao: Implementar `PrintConfiguration()` nos 3 submodulos conforme TODO da Parte 032.

HIGH-11: ReconnectModules pode manter ponteiros stale

Arquivo: `GUI/Panel.mqh`
Severidade: ALTA

Problema: Quando `ReconnectModules()` e chamado (na troca de TF), ele recebe novos ponteiros para todos os modulos. Se algum ponteiro for NULL (falha na recriacao), o sub-painel correspondente mantem o ponteiro antigo (ja freed). O proximo `Update()` do sub-painel pode causar access violation.

Impacto: Crash do EA ao trocar timeframe se algum modulo falhar na reinicializacao.

Correcao: Verificar NULL em cada ponteiro e desabilitar sub-paineis correspondentes se NULL.

HIGH-12: Array bounds potencial em tab buttons

Arquivo: `GUI/PanelTabEstrategias.mqh`, `GUI/PanelTabFiltros.mqh`
Severidade: ALTA

Problema: O codigo usa `m_e_stratBtns[i+1]` onde `i` pode chegar ate `MAX_STRAT_PANELS - 1`, acessando `m_e_stratBtns[MAX_STRAT_PANELS]` que esta fora dos limites do array.

Impacto: Potencial crash ou corrupcao de memoria.

Correcao: Verificar bounds antes de acessar o array com `i+1`.

4. Problemas de Severidade Media

MED-01: `static int lastDay = 0` nao sobrevive a reinicializacao

Arquivo: `EPBot_Matrix.mq5`
Linha: 627
Severidade: MEDIA

Problema: A deteccao de mudanca de dia usa uma variavel static local:

```
static int lastDay = 0;
```

Na primeira execucao do EA, `lastDay = 0`, entao o primeiro tick nunca dispara `ResetDaily()` (a condicao `lastDay != 0` e falsa). Isso e correto para o primeiro dia. Porem, se o EA for reiniciado no meio do dia (recompile, crash), o `lastDay` volta a 0 e o reset diario nao e detectado ate o proximo dia.

Impacto: Se houver posicoes parciais ou contadores de streak do dia anterior persistidos no Logger CSV, eles nao serao resetados apos reinicio intra-dia. Na pratica, `Init()` ja configura os modulos do zero, entao o impacto e baixo.

Correcao: Inicializar `lastDay` com o dia atual na primeira execucao em vez de 0.

MED-02: Chamadas repetidas a SymbolInfoDouble no mesmo tick

Arquivo: `EPBot_Matrix.mq5`, `Core/RiskManager.mqh`
Severidade: MEDIA

Problema: Em um unico tick, `SymbolInfoDouble(_Symbol, SYMBOL_BID)` e `SYMBOL_ASK` sao chamados multiplas vezes:

- `OnTick()` linhas 784-786 (para closePrice)
- `ManageOpenPosition()` linhas 940-942 (para currentPrice)
- `CalculateTrailing()` chama `SymbolInfoInteger(SYMBOL_SPREAD)` e `SymbolInfoDouble(SYMBOL_POINT)`

- `CalculateBreakeven()` idem
- `CalculateSLPrice()`, `CalculateTPPrice()` idem

Cada chamada `SymbolInfoDouble` faz uma syscall interna.

Impacto: Performance sub-otima. Em testes de stress (backtesting), as chamadas redundantes podem impactar a velocidade de otimizacao.

Correcao: Cachear `bid`, `ask`, `point`, `spread` no inicio do `OnTick()` e passar como parametro ou armazenar em uma struct de contexto.

MED-03: Fallback de posicao usa janela de 5 segundos

Arquivo: `EPBot_Matrix.mq5`
Linhas: 1360
Severidade: MEDIA

Problema:

```
if(TimeCurrent() - openTime < 5)
```

O fallback para encontrar a posicao apos `OrderSend` verifica se a posicao foi aberta nos ultimos 5 segundos. Em mercados com alta latencia ou durante periodos de alta volatilidade, a posicao pode demorar mais de 5 segundos para aparecer. Alem disso, `TimeCurrent()` retorna o horario do ultimo tick, nao o horario real.

Impacto: Em cenarios de alta latencia, o EA pode nao encontrar a posicao recen-aberta e falhar em registra-la no TradeManager.

Correcao: Usar `TimeLocal()` ou `GetTickCount()` para comparacao temporal mais precisa, e/ou aumentar a janela.

MED-04: Logger.GetDailyProfit() inclui partialTPProfit que pode contar duas vezes

Arquivo: `Core/Logger.mqh`
Linha: 231
Severidade: MEDIA

Problema:

```
double GetDailyProfit() { return m_dailyProfit + m_partialTPProfit; }
```

O `m_dailyProfit` e atualizado por `UpdateStats(totalPositionProfit)` quando a posicao fecha. O `totalPositionProfit` ja inclui o lucro dos deals parciais (TP1+TP2) pois soma todos os DEAL_ENTRY_OUT. O `m_partialTPProfit` e adicionado separadamente via `AddPartialTPProfit()`.

Quando a posicao fecha completamente, `totalPositionProfit` inclui TP1+TP2+deal final. Se `UpdateStats(totalPositionProfit)` soma isso ao `m_dailyProfit`, e `m_partialTPProfit` ja foi somado antes, ha contagem dupla dos lucros parciais.

Porem, `UpdateStats()` nao soma `totalPositionProfit` ao `m_dailyProfit` diretamente - ele chama `SaveTrade()` que salva no CSV. O `m_dailyProfit` e recalculado via `LoadDailyStats()` que le o CSV. Precisa verificar se `LoadDailyStats()` soma ou nao os parciais.

Impacto: Potencial contagem dupla de lucros parciais no `GetDailyProfit()`, afetando limites diarios e protecao de drawdown.

Correcao: Revisar o fluxo completo de contabilizacao para garantir que parciais nao sejam contados duas vezes. O ideal e que `m_dailyProfit` contenha TUDO (incluindo parciais) e `m_partialTPProfit` seja removido.

MED-05: Div por zero possivel em GetCurrentDrawdown

Arquivo: `Core/BlockerDrawdown.mqh`
Linhas: 677-709
Severidade: MEDIA

Problema: Em `GetCurrentDrawdown()`:

```
if(!m_drawdownProtectionActive || m_dailyPeakProfit <= 0)
   return 0.0;
```

O guard protege contra pico <= 0. Porem, se `m_drawdownType == DD_PERCENTAGE`:

```
return (currentDD / m_dailyPeakProfit) * 100.0;
```

Embora o guard proteja contra `m_dailyPeakProfit <= 0`, ha um path onde `m_dailyPeakProfit` pode se tornar muito proximo de zero (float) sem ser exatamente <= 0, causando um resultado muito grande.

Impacto: Valor de drawdown absurdamente grande exibido na GUI, confundindo o operador.

Correcao: Adicionar guard com epsilon: `if(m_dailyPeakProfit < 0.01) return 0.0;`

MED-06: `CalculateAverageRange()` nao validado

Arquivo: `Core/RiskManager.mqh`
Severidade: MEDIA

Problema: O metodo `CalculateAverageRange()` nao foi verificado nesta analise por nao estar na parte lida do arquivo, mas e usado em `CalculateSLPrice()` para SL_RANGE. Se retornar 0 ou valor negativo, `slDistance` sera 0 ou negativo, e o SL ficara no preco de entrada ou alem dele.

Impacto: SL invalido (muito proximo ou no preco errado) em modo SL_RANGE.

Correcao: Garantir que `CalculateAverageRange()` tenha guard contra retorno <= 0, com fallback para SL fixo.

MED-07: Sem validacao de MaxSpread contra spread negativo

Arquivo: `Core/BlockerFilters.mqh`
Severidade: MEDIA

Problema: `CheckSpreadFilter()` compara `currentSpread >= m_maxSpread`. Em instrumentos CFD ou durante erros de dados, o spread pode vir como negativo. A comparacao funciona corretamente (negativo < qualquer maxSpread positivo), mas nao ha log de alerta para spread negativo, que seria um indicador de problema de dados.

Impacto: Baixo, mas spread negativo silencioso pode mascarar problemas de conexao.

Correcao: Logar alerta quando spread < 0.

MED-08: Timer nao destruido em todos os paths de OnDeinit

Arquivo: `EPBot_Matrix.mq5`
Linhas: 560-574
Severidade: MEDIA

Problema: No `OnDeinit`, quando `reason == REASON_CHARTCHANGE`, o painel e mantido vivo. O timer e morto com `EventKillTimer()` na linha 574. Porem, o comentario diz "Apenas mata o timer; painel e objetos graficos preservados", mas o `EventKillTimer()` esta FORA do bloco `if(g_panel != NULL)`, entao ele executa para todos os reasons. Isso esta correto.

Porem, na `OnInit`, se o painel nao existir e a reinit for por `REASON_CHARTCHANGE`, o `ReconnectModules()` e chamado e o timer e re-setado (linha 434). Se `CreatePanel()` falhar, o timer nao e setado mas pode haver um timer residual de antes.

Impacto: Baixo - timer orfao que chama `g_panel.Update()` com `g_panel == NULL`.

Correcao: O guard em `OnTimer` deve verificar `g_panel != NULL` (e provavelmente ja faz no Update interno).

MED-09: `GetTickCount()` overflow de 49 dias

Arquivo: `GUI/*.mqh` (multiplos)
Severidade: MEDIA

Problema: Varios paineis usam `GetTickCount()` para timers de expiracao (ex: `m_statusExpiry`). `GetTickCount()` retorna uint que sofre overflow a cada ~49 dias. Apos overflow, comparacoes de tempo podem produzir resultados incorretos.

Impacto: Apos ~49 dias de execucao continua, banners e mensagens de status podem nao expirar corretamente. Improvavel em producao (EA geralmente reinicia mais frequentemente).

Correcao: Usar `GetTickCount64()` (disponivel desde MT5 build 2361) ou tratar o overflow.

MED-10: `CycleAppliedPrice` omite PRICE_WEIGHTED

Arquivo: `GUI/PanelUtils.mqh`
Severidade: MEDIA

Problema: A funcao `CycleAppliedPrice()` cicla entre os precos disponiveis mas omite `PRICE_WEIGHTED` (preco ponderado). O usuario nunca pode selecionar este tipo de preco via GUI.

Impacto: Funcionalidade reduzida - nem todos os precos aplicados sao acessiveis via GUI.

Correcao: Adicionar `PRICE_WEIGHTED` ao ciclo.

MED-11: News filter: codigo triplicado

Arquivo: `GUI/PanelTabConfig.mqh`
Severidade: MEDIA

Problema: O codigo para 3 janelas de noticias e copy-pasted 3 vezes com diferencas minimas (apenas indices 1/2/3). Sao ~200 linhas duplicadas.

Impacto: Manutencao dificil - bugs corrigidos em uma janela podem nao ser corrigidos nas outras.

Correcao: Extrair logica comum em funcao parametrizada por indice da janela.

MED-12: SetEnabled(false) sobrescreve CLR_FIELD_ERROR

Arquivo: `GUI/FilterPanelBase.mqh`, `GUI/StrategyPanelBase.mqh`
Severidade: MEDIA

Problema: Quando um sub-painel e desabilitado, `SetEnabled(false)` define a cor dos campos como cinza, sobrescrevendo o rosa (`CLR_FIELD_ERROR`) de campos invalidos. Quando o painel e re-habilitado, o erro visual se perde.

Impacto: Usuario perde feedback visual de validacao ao alternar o toggle ON/OFF de uma estrategia/filtro.

Correcao: `SetEnabled(true)` deveria preservar ou re-aplicar `CLR_FIELD_ERROR` dos campos que ainda estao invalidos.

MED-13: ValidateSLTP sempre retorna is_valid=true

Arquivo: `Core/RiskManager.mqh`
Linhas: ~1310-1474
Severidade: MEDIA

Problema: O metodo `ValidateSLTP()` nunca retorna `is_valid = false`. Ele sempre ajusta os precos para respeitar o stop level do broker, mas nunca rejeita. Isso significa que o chamador pode confiar em `is_valid` sem checar os campos `sl_adjusted`/`tp_adjusted`.

Impacto: Nao e um bug per se, mas cria uma API enganosa. Se um futuro desenvolvedor esperar que `is_valid=false` signifique "nao pode operar", nunca sera atingido.

Correcao: Ou remover `is_valid` (sempre true) ou documentar claramente que a funcao NUNCA rejeita, apenas ajusta.

MED-14: Sem retry em OrderSend para parciais

Arquivo: `Core/TradeManager.mqh`
Linhas: 759-842 (ExecutePartialClose)
Severidade: MEDIA

Problema: Se o `OrderSend` para TP1 ou TP2 falhar (requote, off quotes), nao ha retry. O TP parcial e simplesmente marcado como "nao executado" e o proximo tick tentara novamente (pois `tp1_executed` nao foi setado). Isso e aceitavel para requotes temporarios, mas se houver um problema persistente (volume invalido, simbolo suspenso), o EA tentara a cada tick.

Impacto: Spam de OrderSend em caso de falha persistente. Porem, como `ExecutePartialClose` retorna false e o flag nao e setado, o retry implicito via MonitorPartialTP funciona como mecanismo de retry natural.

Correcao: Adicionar cooldown entre tentativas de TP parcial (ex: 5 segundos entre retries).

MED-15: CalculateTPPrice nao valida contra stop level

Arquivo: `Core/RiskManager.mqh`
Linhas: ~1257-1308
Severidade: MEDIA

Problema: Ao contrario de `CalculateSLPrice()` que clamp o SL ao stop level minimo do broker, `CalculateTPPrice()` nao faz essa validacao. Se o TP calculado (especialmente com ATR baixo ou spread compensation alta) resultar em uma distancia menor que o stop level, o OrderSend falhara.

A validacao e feita depois em `ValidateSLTP()`, mas somente se o chamador usar esse metodo (o que o `ExecuteTrade` faz, mas parciais nao passam por ela).

Impacto: Parciais com TP muito proximo podem ter preco invalido.

Correcao: Adicionar validacao de stop level em `CalculateTPPrice()` ou garantir que `CalculatePartialTPLevels()` tambem valide.

MED-16: Netting mode nao testado

Arquivo: Todos
Severidade: MEDIA

Problema: O EA assume modelo de conta hedging em varios pontos:

- `PositionGetTicket(i)` retorna tickets unicos por posicao (hedging)
- Detecta "multiplas posicoes" como erro (em netting, so pode ter uma por simbolo)
- `HistorySelectByPosition` com ticket
- Partial close via `TRADE_ACTION_DEAL` com volume parcial

Em contas netting:

- So pode existir uma posicao por simbolo (o alerta de duplicata nunca dispararia)
- Partial close funciona diferente (reduz o volume da posicao unica)
- Position ticket muda apos cada operacao parcial

Impacto: O EA pode ter comportamento inesperado em contas netting. Parciais e state tracking podem falhar.

Correcao: Adicionar deteccao de `ACCOUNT_MARGIN_MODE` no Init e alertar ou rejeitar modo netting. Ou implementar suporte explicito para netting.

MED-17: Fechamento por drawdown pode causar loop com TP parcial

Arquivo: `EPBot_Matrix.mq5`
Linhas: 997-1038
Severidade: MEDIA

Problema: Se `ShouldCloseByDrawdown()` retorna true e o `OrderSend` fecha a posicao, no proximo tick a posicao nao existira mais e o fluxo normal de deteccao de fechamento sera ativado. Porem, se a posicao teve TP1 executado (volume reduzido), o `PositionGetDouble(POSITION_VOLUME)` na request de fechamento usa o volume ATUAL (correto). Mas se entre o TP1 e o fechamento por drawdown houve slippage que alterou o volume registrado, pode haver discrepancia.

Impacto: Baixo, mas possivel tentativa de fechar volume diferente do real.

Correcao: Re-selecionar a posicao e re-ler o volume imediatamente antes do `OrderSend`.

MED-18: ConfigPersistence nao valida inputs numericos

Arquivo: `Core/ConfigPersistence.mqh`
Severidade: MEDIA

Problema: `CollectConfigData()` le campos de texto da GUI com `StringToInteger()`/`StringToDouble()`. Se o campo contiver texto invalido (ex: "abc" em vez de um numero), o resultado e 0 silenciosamente. O valor 0 e salvo no arquivo de config.

Impacto: Configuracao corrompida se o usuario digitar texto invalido em campos numericos. Na pratica, a validacao do `ApplyConfig()` deveria pegar isso, mas a config ja foi salva em disco.

Correcao: Validar campos antes de salvar, ou pelo menos nao salvar se a validacao falhar.

5. Problemas de Severidade Baixa

LOW-01: Comentario stale "KNOWN LIMITATION"

Arquivo: `EPBot_Matrix.mq5`
Linha: 736
Severidade: BAIXA

Problema: O comentario na linha 736 diz "KNOWN LIMITATION" mas o changelog v1.59 (linhas 61-67) diz que foi corrigido. Ambos existem simultaneamente, gerando confusao.

Correcao: Remover ou atualizar o comentario inline para refletir a correcao.

LOW-02: Assimetria de API StrategyBase vs FilterBase

Arquivo: `Strategy/Base/StrategyBase.mqh`, `Strategy/Base/FilterBase.mqh`
Severidade: BAIXA

Problema: `CStrategyBase` usa `GetEnabled()` enquanto `CFilterBase` usa `IsEnabled()`. Documentado nas notas tecnicas (linha 77-80 do mq5) mas nao corrigido.

Impacto: Inconsistencia de API que dificulta uso polimorfico.

Correcao: Unificar para `GetEnabled()` ou `IsEnabled()` em ambas.

LOW-03: `#property version` inconsistente em Blockers.mqh

Arquivo: `Core/Blockers.mqh`
Linha: 9
Severidade: BAIXA

Problema: `#property version "3.26"` mas o titulo diz "Versao 3.26" e os changelogs mencionam v3.25 e v3.26. Os logs de Init ainda dizem "v3.25".

Correcao: Atualizar string de log para v3.26.

LOW-04: Emoji em logs pode causar problemas em terminais mais antigos

Arquivo: Todos
Severidade: BAIXA

Problema: Uso extensivo de emojis Unicode em mensagens de log. Terminais MT5 mais antigos ou em sistemas com codificacao restrita podem exibir caracteres corrompidos.

Impacto: Logs ilegíveis em ambientes restritos.

Correcao: Usar prefixos textuais como `[OK]`, `[ERR]`, `[WARN]` como alternativa ou fallback.

LOW-05: Multiplos `PositionGetSymbol(i)` em loops

Arquivo: Varios (`EPBot_Matrix.mq5`, `BlockerDrawdown.mqh`)
Severidade: BAIXA

Problema: Em loops como:

```
for(int i = PositionsTotal() - 1; i >= 0; i--)
   if(PositionGetSymbol(i) == _Symbol && ...)
```

`PositionGetSymbol(i)` faz a selecao implicita da posicao. Isso e lento se houver muitas posicoes de outros EAs.

Impacto: Performance levemente reduzida em contas com muitas posicoes.

Correcao: Nenhuma acao necessaria para operacao normal. Otimizar se performance for problema.

LOW-06: `ENUM_SIGNAL_TYPE` usa -1 para SELL

Arquivo: `Strategy/Base/StrategyBase.mqh`
Severidade: BAIXA

Problema: `SIGNAL_SELL = -1` funciona mas e incomum. Comparacoes com `!= 0` podem confundir com `SIGNAL_BUY = 1`.

Impacto: Nenhum bug, mas confuso para novos contribuidores.

Correcao: Documentar a convencao explicitamente.

LOW-07: `SetButtonEnabled()` nao restaura cores

Arquivo: `GUI/PanelUtils.mqh`
Severidade: BAIXA

Problema: `SetButtonEnabled(enable=true)` habilita o botao mas nao restaura suas cores originais. O chamador precisa restaurar as cores manualmente apos chamar a funcao.

Impacto: Contrato implicito que pode levar a botoes com cores erradas se esquecido.

Correcao: Adicionar parametros de cor ou documentar o contrato claramente.

LOW-08: Profit Factor mostra vermelho para valor 0

Arquivo: `GUI/PanelTabResultados.mqh`
Severidade: BAIXA

Problema: Quando `grossLoss == 0` (nenhum trade perdedor), o Profit Factor e 0.0 e exibido em vermelho. Deveria ser exibido como "∞" ou em cor neutra.

Correcao: Tratar caso especial `grossLoss == 0`.

LOW-09: Dead code `_placeholder_private_end`

Arquivo: `GUI/Panels/BollingerBandsPanel.mqh`
Severidade: BAIXA

Problema: Campo placeholder nao utilizado.

Correcao: Remover.

LOW-10: Variavel `bg` morta em MACrossPanel::SetEnabled

Arquivo: `GUI/Panels/MACrossPanel.mqh`
Severidade: BAIXA

Problema: Variavel declarada mas nao utilizada.

Correcao: Remover.

LOW-11: Hardcoded color literals

Arquivo: Todos os arquivos GUI
Severidade: BAIXA

Problema: Cores como `C'30,120,70'`, `C'120,50,50'`, `C'50,80,140'` aparecem centenas de vezes inline. Nao ha constantes nomeadas.

Impacto: Dificil mudar o tema visual. Risco de inconsistencia de cores entre paineis.

Correcao: Definir constantes de cor centralizadas em `PanelUtils.mqh`.

LOW-12: `ESTRAT_PAGE_COUNT` e `FILTROS_PAGE_COUNT` hardcoded

Arquivo: `GUI/Panel.mqh`
Severidade: BAIXA

Problema: Constantes fixas em 3. Adicionar uma nova estrategia ou filtro requer atualizar manualmente essa constante.

Correcao: Derivar do numero real de paineis registrados.

LOW-13: `RSIFilterPanel::Apply()` nao chama `SetAppliedPrice`

Arquivo: `GUI/Panels/RSIFilterPanel.mqh`
Severidade: BAIXA

Problema: O painel permite ciclar o `AppliedPrice`, mas o `Apply()` nao envia o valor ao filtro.

Impacto: Hot reload do applied price do RSI Filter via GUI nao funciona.

Correcao: Adicionar chamada ao setter correspondente.

LOW-14: `BollingerBandsFilterPanel::Show()` nao chama `_RefreshFieldState()`

Arquivo: `GUI/Panels/BollingerBandsFilterPanel.mqh`
Severidade: BAIXA

Problema: Ao re-exibir o painel, os estados dos campos (habilitados/desabilitados baseado no modo selecionado) nao sao recalculados.

Impacto: Campos em estado visual incorreto ao navegar entre abas.

Correcao: Chamar `_RefreshFieldState()` ao final de `Show()`.

6. Problemas de Arquitetura

ARCH-01: Acoplamento excessivo via ponteiros globais

Severidade: MEDIA

Todos os modulos (Logger, Blockers, RiskManager, TradeManager, SignalManager, Strategies, Filters, Panel) sao acessados via ponteiros globais `g_*`. Isso cria acoplamento forte e dificulta testes unitarios.

Sugestao: Para o contexto de MQL5 (sem dependency injection nativo), o padrao e aceitavel. Documentar a hierarquia de dependencias.

ARCH-02: Responsabilidades misturadas em ManageOpenPosition

Severidade: MEDIA

A funcao `ManageOpenPosition()` (~280 linhas) mistura:

- Verificacao de limites diarios
- Protecao de drawdown
- Monitoria de TP parcial
- Trailing stop
- Breakeven
- Sinal de saida
- Virar a mao (VM)

Sugestao: Extrair cada responsabilidade em funcoes menores (ex: `CheckDailyLimitClose()`, `CheckDrawdownClose()`, `ManageTrailing()`, etc.).

ARCH-03: Duplicacao entre GUI tabs

Severidade: BAIXA

`PanelTabEstrategias.mqh` e `PanelTabFiltros.mqh` sao praticamente identicos estruturalmente. Poderiam compartilhar uma classe base `PanelTabPagedBase`.

ARCH-04: Init() com muitos parametros

Severidade: BAIXA

`CBlockers::Init()` recebe 32 parametros. Considerar usar struct de configuracao (como `SConfigData` ja existe para persistencia).

7. Resumo por Categoria

Categoria	Critico	Alto	Medio	Baixo
Erros de Logica	2	2	3	2
Bugs e Falhas	1	3	4	3
Problemas de Trading	1	2	4	0
Performance	0	0	2	2
GUI/UX	0	3	4	5

Categoria	Critico	Alto	Medio	Baixo
Arquitetura	0	2	1	2
Total	4	12	18	14

Arquivos Analisados (35 total)

Core (8 arquivos)

- `EPBot_Matrix.mq5` - EA principal (~1474 linhas)
- `Core/Inputs.mqh` - Inputs centralizados (~305 linhas)
- `Core/Logger.mqh` - Sistema de logging (~1143 linhas)
- `Core/Blockers.mqh` - Orchestrator de bloqueios (~813 linhas)
- `Core/BlockerFilters.mqh` - Filtros de mercado (~981 linhas)
- `Core/BlockerDrawdown.mqh` - Protecao drawdown (~712 linhas)
- `Core/BlockerLimits.mqh` - Limites diarios/streak (~925 linhas)
- `Core/RiskManager.mqh` - Gestao de risco (~1993 linhas)
- `Core/TradeManager.mqh` - Gestao de posicoes (~1227 linhas)
- `Core/Utils.mqh` - Utilitarios (~32 linhas)
- `Core/ConfigPersistence.mqh` - Persistencia de config (~737 linhas)

Strategy (6 arquivos)

- `Strategy/Base/StrategyBase.mqh` (~129 linhas)
- `Strategy/Base/FilterBase.mqh` (~91 linhas)
- `Strategy/SignalManager.mqh` (~970 linhas)
- `Strategy/Strategies/MACrossStrategy.mqh` (~979 linhas)
- `Strategy/Strategies/RSIStrategy.mqh` (~714 linhas)
- `Strategy/Strategies/BollingerBandsStrategy.mqh` (~900 linhas)

Filters (3 arquivos)

- `Strategy/Filters/TrendFilter.mqh` (~861 linhas)
- `Strategy/Filters/RSIFilter.mqh` (~730 linhas)
- `Strategy/Filters/BollingerBandsFilter.mqh` (~694 linhas)

GUI (16 arquivos)

- `GUI/Panel.mqh`
- `GUI/PanelPersistence.mqh`
- `GUI/PanelTabConfig.mqh`
- `GUI/PanelTabEstrategias.mqh`
- `GUI/PanelTabFiltros.mqh`
- `GUI/PanelTabResultados.mqh`
- `GUI/PanelTabStatus.mqh`
- `GUI/PanelUtils.mqh`
- `GUI/FilterPanelBase.mqh`
- `GUI/StrategyPanelBase.mqh`
- `GUI/Panels/BollingerBandsFilterPanel.mqh`
- `GUI/Panels/BollingerBandsPanel.mqh`
- `GUI/Panels/MACrossPanel.mqh`
- `GUI/Panels/RSIFilterPanel.mqh`
- `GUI/Panels/RSIStrategyPanel.mqh`
- `GUI/Panels/TrendFilterPanel.mqh`

Fim da analise. Gerado em 2026-04-03 por Claude Opus 4.6.